

Milestone 1: Literature Review and Sequential Baseline for Canny Edge Detection

Anjali Kamath

Department of Computer Science

Your University

Email: your.email@example.com

Abstract—This milestone establishes a sequential reference implementation for Canny edge detection as the baseline for subsequent parallelization. The work is based primarily on Horvath, Bowers, and Alawneh’s CUDA implementation of Canny edge detection. The algorithm is decomposed into grayscale conversion, Gaussian smoothing, Sobel gradient computation, non-maximum suppression, and double thresholding with hysteresis. A single-threaded C implementation is developed using a simple PPM/PGM image interface and is instrumented with per-stage timing. The baseline provides both a correctness reference and a performance denominator for calculating speedup in subsequent parallel implementations.

I. INTRODUCTION

Edge detection is a fundamental preprocessing operation in computer vision. The Canny edge detector is particularly useful because it combines noise reduction, gradient estimation, edge thinning, and edge connectivity into a multi-stage pipeline. The reference paper studied Canny edge detection as a GPU acceleration problem and compared a sequential C implementation, a multithreaded CPU implementation using oneTBB, and a CUDA implementation using tiled shared memory [?].

The objective of this milestone is to establish a correct and measurable single-threaded implementation before introducing parallelism. The sequential implementation serves as the reference against which later CPU or GPU implementations will be validated and benchmarked.

II. LITERATURE REVIEW

A. Parallel Canny Edge Detection

Canny edge detection has been investigated on several parallel architectures. Ben Cheikh et al. studied parallelization strategies for the Canny detector using multicore CPU and many-core GPU architectures, including OpenMP and CUDA approaches [?]. This work motivates treating the per-pixel operations in the Canny pipeline as candidates for loop-level parallelism.

Rahamneh and Sawalha investigated a heterogeneous CPU-FPGA implementation using OpenCL and image tiling with load balancing [?]. Their approach demonstrates that domain decomposition can be used to distribute image processing work across heterogeneous processing resources.

Luo and Duraiswami presented an early CUDA implementation of Canny edge detection and demonstrated substantial acceleration relative to CPU-based implementations [?]. Their work is particularly relevant because it established GPU execution as a practical approach for accelerating the Canny pipeline.

B. Primary Reference

The primary reference for this project is Horvath, Bowers, and Alawneh, “Canny Edge Detection on GPU using CUDA” [?]. The authors implemented three versions of the algorithm: a sequential C implementation, a TBB-based CPU implementation, and a CUDA implementation. Their CUDA version uses a separate kernel for each computational stage and employs shared-memory tiling for convolutional stages.

The paper reports measurements from 40×30 through 3840×2160 (4K) resolutions. For the sequential implementation, the reported total time at 4K is approximately 1482.5 ms, while the CUDA implementation takes approximately 14.25 ms. This corresponds to a reported improvement of more than $100\times$ at 4K [?]. These values are useful as reference results, but they should not be treated as expected runtimes for the present system because hardware and implementation environments differ.

The paper also identifies important optimization opportunities. Its Sobel kernel was found to be a significant bottleneck, with the magnitude and direction calculations involving $\text{hypot}/\text{atan2}$ -type operations contributing substantially to the computation. The authors also discuss memory-access behavior and CUDA streams as possible areas for future optimization [?].

III. PROBLEM DEFINITION

The target problem is to compute an edge map from an RGB image using the Canny edge detection algorithm. The sequential reference follows the logical pipeline

$$\text{RGB} \rightarrow \text{Grayscale} \rightarrow \text{Gaussian Blur} \rightarrow \text{Sobel} \rightarrow \text{NMS} \rightarrow \text{Threshold} + \text{Hysteresis} \quad (1)$$

The primary objective of the baseline is not optimization. It is to produce a correct, reproducible implementation whose runtime can be used as the reference value

$$T_{\text{seq}} \quad (2)$$

for future speedup measurements.

IV. CANNY EDGE DETECTION ALGORITHM

A. RGB to Grayscale

Each RGB pixel is converted into one grayscale intensity using the luminosity method:

$$I(x, y) = 0.299R(x, y) + 0.587G(x, y) + 0.114B(x, y). \quad (3)$$

The operation is independent for each pixel, making this stage naturally amenable to parallel execution.

B. Gaussian Blur

The grayscale image is smoothed using the 5×5 Gaussian kernel given in the reference paper:

$$K_G = \frac{1}{150} \begin{bmatrix} 2 & 4 & 5 & 4 & 2 \\ 4 & 9 & 12 & 9 & 4 \\ 5 & 12 & 15 & 12 & 5 \\ 4 & 9 & 12 & 9 & 4 \\ 2 & 4 & 5 & 4 & 2 \end{bmatrix}. \quad (4)$$

The coefficients sum to 150, so the kernel is normalized. A direct single-threaded convolution is used in the baseline. Pixels outside the image boundary are treated as zero, consistent with the boundary behavior described for the reference CUDA implementation [?].

C. Sobel Filtering

The Sobel operator estimates the horizontal and vertical image gradients. The kernels are

$$S_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}, \quad S_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}. \quad (5)$$

For each pixel, the resulting gradient magnitude and direction are

$$M(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2}, \quad (6)$$

$$\theta(x, y) = \text{atan2}(G_y(x, y), G_x(x, y)). \quad (7)$$

The reference paper describes the same magnitude and direction calculation using the hypotenuse and arctangent of the two Sobel components [?].

D. Non-Maximum Suppression

Non-maximum suppression thins gradient responses. For each pixel, its gradient direction is quantized into one of four orientations: 0° , 45° , 90° , or 135° . The pixel is retained only when its magnitude is at least as large as the two neighboring pixels along that direction.

This stage removes non-maximal responses while retaining local edge peaks. The exact numerical direction-quantization rule is not fully specified in the reference paper; therefore, the four-orientation quantization used here is explicitly treated as an implementation choice for the sequential reference.

E. Double Thresholding

After non-maximum suppression, pixels are classified using a low and high threshold. The baseline uses configurable values with defaults

$$T_L = 50, \quad T_H = 100. \quad (8)$$

The classification is

$$C(x, y) = \begin{cases} 255, & M(x, y) \geq T_H, \\ 127, & T_L \leq M(x, y) < T_H, \\ 0, & M(x, y) < T_L. \end{cases} \quad (9)$$

The reference paper uses 255 for strong pixels, 127 for weak pixels, and 0 for non-edge pixels [?]. The paper does not specify numerical low and high threshold values in the methodology, so the values above are parameters of the present baseline rather than values claimed to come directly from the paper.

F. Hysteresis

Hysteresis determines which weak responses belong to actual edges. The baseline performs an 8-connected graph traversal starting from all strong pixels. Weak pixels connected to a strong component are promoted to strong edges; remaining weak pixels are discarded.

This stack-based flood-fill is an implementation choice for the sequential reference. It also illustrates an important distinction for future parallelization: unlike the preceding pixel-local stages, edge connectivity introduces dependencies between neighboring pixels.

G. Test Input Generation

For the initial correctness test, the following Python script was used to generate the synthetic P6 PPM input. It creates a black background containing a white rectangle and a white circle.

```
from pathlib import Path

w, h = 160, 120
with open("input.ppm", "wb") as f:
    f.write(f"P6\n{w} {h}\n255\n".encode())

    for y in range(h):
        for x in range(w):
            r = g = b = 0

            # white rectangle
```

```

if 20 <= x < 100 and 20 <= y < 80:
    r = g = b = 255

# white circle
elif (x - 125)**2 + (y - 60)**2 < 25**2:
    r = g = b = 255

f.write(bytes([r, g, b]))

```

V. SEQUENTIAL BASELINE IMPLEMENTATION

The reference implementation is written in standard C and is intentionally single-threaded. No OpenMP, CUDA, TBB, SIMD intrinsics, or other explicit parallelism is used.

The implementation uses binary PPM (P6) input and binary PGM (P5) output. This follows the reference paper’s use of PPM files during its Linux-based development and testing environment [?]. Keeping image I/O simple also avoids introducing an external image-processing library into the computational baseline.

The implementation consists of the following functions:

- `rgb_to_grayscale()` — Stage 1.
- `gaussian_blur()` — Stage 2.
- `sobel()` — Stage 3.
- `non_maximum_suppression()` — Stage 4.
- `double_threshold_and_hysteresis()` — Stage 5.

Each image is stored as a flattened one-dimensional array. A pixel at coordinate (x, y) is accessed using

$$\text{index} = yW + x, \quad (10)$$

where W is the image width. This layout is deliberately simple and maps naturally to the one-thread-per-output-pixel structure that can be used in a later CUDA implementation.

VI. PERFORMANCE METRICS

The primary performance metric is end-to-end computation time:

$$T_{\text{seq}} = T_G + T_B + T_S + T_{\text{NMS}} + T_H, \quad (11)$$

where the terms correspond to grayscale, Gaussian blur, Sobel, non-maximum suppression, and thresholding plus hysteresis.

Each stage is timed independently using a monotonic wall-clock timer. Image loading and output writing are excluded from the algorithm timing. This prevents file-system I/O from obscuring the computational cost of the Canny pipeline.

The following secondary metrics will also be collected:

- 1) execution time for each stage;
- 2) total execution time;
- 3) execution time as a function of image resolution;
- 4) average runtime over repeated executions;
- 5) later, parallel speedup relative to the sequential baseline.

For a future parallel implementation,

$$\text{Speedup} = \frac{T_{\text{seq}}}{T_{\text{parallel}}}. \quad (12)$$

The reference paper benchmarks resolutions from 40×30 through 3840×2160 , so the same resolution set is proposed for this project where runtime permits [?].

VII. EXPERIMENTAL PLAN

The benchmark matrix is

TABLE I
PLANNED SEQUENTIAL BASELINE MEASUREMENTS.

Resolution	Gray	Gaussian	Sobel	NMS	Hyst.	Total
40×30	–	–	–	–	–	–
640×480	–	–	–	–	–	–
1280×720	–	–	–	–	–	–
1920×1080	–	–	–	–	–	–
3840×2160	–	–	–	–	–	–

Each configuration should be executed repeatedly and averaged. Both `-O0` and `-O2` compiler settings can be recorded separately. The unoptimized result provides a simple naive reference, while the optimized result provides a fairer CPU baseline for eventual comparison against optimized parallel code.

The primary reported comparison should use the same compiler optimization policy for all CPU baselines. Exact runtimes from the reference paper are not treated as expected values because its experiments used different hardware, compilers, and implementation details.

A. Observed Test Run

A synthetic 160×120 PPM test image was generated containing a white rectangle and a white circle on a black background. This input was chosen because the expected geometric edges are known in advance, making it useful for a first visual correctness check.

The test was executed with the default thresholds $T_L = 50$ and $T_H = 100$ using the unoptimized sequential build (`-O0`). The measured stage times were:

TABLE II
OBSERVED SEQUENTIAL BASELINE RUN ON THE 160×120 TEST IMAGE.

Stage	Time (ms)
Grayscale	0.100
Gaussian Blur	3.896
Sobel Filter	2.933
NMS	0.186
Threshold + Hysteresis	0.200
Total	7.315

The corresponding terminal output was:

```

Resolution: 160x120
Thresholds: low=50.00, high=100.00

Stage Time (ms)
-----
Grayscale 0.100
Gaussian Blur 3.896
Sobel Filter 2.933
NMS 0.186
Threshold + Hysteresis 0.200

```

The resulting edge map is shown in Fig. ?? . The rectangle and circular boundary are clearly recovered as thin edge structures. Small breaks and isolated pixels are visible around portions of the boundaries; these are a consequence of the discrete convolution, thresholding, and non-maximum-suppression operations rather than an I/O failure. The output therefore provides a useful qualitative sanity check before larger benchmark runs.

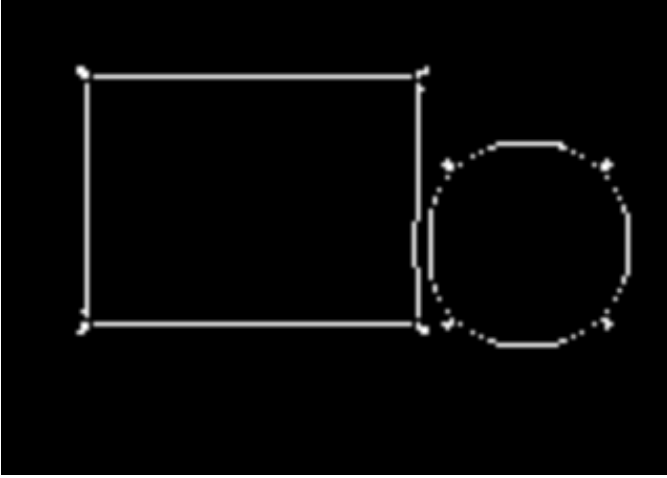


Fig. 1. Final edge map produced by the sequential Canny baseline for the synthetic 160×120 input image.

VIII. CORRECTNESS VALIDATION

Correctness is evaluated at multiple levels.

First, the program must preserve the input dimensions and produce a valid 8-bit output edge map. Second, intermediate images can be written and visually inspected after grayscale conversion, Gaussian filtering, Sobel processing, and non-maximum suppression. Third, the final edge map can be compared against an independently implemented reference, such as a Python implementation, to identify algorithmic discrepancies before parallelization.

A preliminary 160×120 synthetic-image test produced the expected geometric edge structure. The measured result and visual inspection are reported in the observed test-run subsection below; this is a correctness sanity check rather than a statistically repeated performance benchmark.

IX. PARALLELIZATION OPPORTUNITIES

The reference paper observes that pixels within a given Canny stage can be computed independently, making the algorithm suitable for parallel execution [?].

TABLE III
PARALLELIZATION CHARACTERISTICS OF THE LOGICAL STAGES.

Stage	Parallelization characteristic
Grayscale	Each output pixel depends only on its own RGB values. Very high parallelism.
Gaussian	Each output pixel depends on a fixed local 5×5 neighborhood. Suitable for one-output-pixel-per-thread execution and tiling.
Sobel	Each output pixel uses a fixed local 3×3 neighborhood. Highly parallelizable; memory access and mathematical operations are important performance considerations.
NMS	Each output pixel examines its local neighbors along its gradient direction. The operation is still pixel-local and highly parallelizable.
Threshold	Each output pixel is classified independently. Very high parallelism.
Hysteresis	Weak-edge connectivity introduces neighborhood-dependent propagation, making this stage substantially more difficult to parallelize efficiently.

This decomposition provides a direct starting point for a CUDA implementation. In particular, the grayscale, Gaussian, Sobel, and NMS loops can be mapped to GPU threads, while Gaussian and Sobel are natural candidates for shared-memory tiling.

The reference paper uses 16×16 CUDA thread blocks and shared-memory tiling for its convolution stages [?]. Its profiling also identifies Sobel as an important bottleneck and discusses memory access and the cost of gradient magnitude and direction calculations as optimization targets.

X. CONCLUSION

This milestone establishes the sequential reference implementation required before parallelization. The implementation follows the Canny pipeline used by the primary reference and uses a straightforward single-threaded C implementation with explicit per-stage timing.

The resulting baseline provides two essential resources for subsequent milestones: a correctness reference and a quantitative performance denominator. Future parallel implementations can be validated against the same output and evaluated using speedup,

$$\text{Speedup} = \frac{T_{\text{seq}}}{T_{\text{parallel}}}. \quad (13)$$

The most promising parallel targets are the pixel-local grayscale, convolution, Sobel, NMS, and thresholding stages. Hysteresis presents a more challenging connectivity problem and will require additional consideration during the parallel implementation phase.

IMPLEMENTATION NOTES

The supplied source file is intended to be compiled as a standalone C program. The default threshold values are 50 and 100 but can be changed on the command line.

```
clang -std=c11 -O0 -Wall -Wextra \
    -o canny_baseline canny_baseline.c -lm
./canny_baseline input.ppm output.pgm 50 100
```

For an optimized CPU baseline:

```
clang -std=c11 -O2 -Wall -Wextra \
-o canny_baseline_O2 canny_baseline.c -lm
```

APPENDIX: SEQUENTIAL C SOURCE CODE

The complete source used for the measured run is included below. The source implements the five-stage Canny pipeline as a single-threaded C program, including PPM input, PGM output, per-stage timing, zero-padded convolution, Sobel magnitude and direction, non-maximum suppression, and stack-based 8-connected hysteresis. The source file is also supplied separately with the submission.

```
/*
 * Sequential single-threaded C reference implementation of Canny edge detection.
 *
 * Based on:
 * M. Horvath Jr., M. Bowers, and S. Alawneh,
 * "Canny Edge Detection on GPU using CUDA,"
 * IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC),
 * 2023.
 *
 * Logical stages:
 * 1. RGB -> grayscale
 * 2. 5x5 Gaussian blur
 * 3. Sobel X/Y -> magnitude + direction
 * 4. Non-maximum suppression
 * 5. Double threshold + hysteresis
 *
 * I/O:
 * Input : binary PPM (P6)
 * Output : binary PGM (P5)
 *
 * Build:
 * clang -std=c11 -O0 -Wall -Wextra -o canny_baseline canny_baseline.c -lm
 * clang -std=c11 -O2 -Wall -Wextra -o canny_baseline_O2 canny_baseline.c -lm
 *
 * Usage:
 * ./canny_baseline input.ppm output.pgm [low_threshold] [high_threshold]
 *
 * Example:
 * ./canny_baseline input.ppm output.pgm 50 100
 */

#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <time.h>
#include <limits.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

typedef struct {
    int width;
    int height;
    unsigned char *r;
    unsigned char *g;
    unsigned char *b;
} RGBImage;

typedef struct {
    int width;
    int height;
    double *data;
} GrayImage;

/* ----- Utilities ----- */

static void die(const char *message)
{
    fprintf(stderr, "Error: %s\n", message);
    exit(EXIT_FAILURE);
}

static void *xmalloc(size_t n)
{
    void *p = malloc(n);
    if (!p) die("memory allocation failed");
    return p;
}

static void *xcalloc(size_t count, size_t size)
{
    void *p = calloc(count, size);
    if (!p) die("memory allocation failed");
    return p;
}

static double now_ms(void)
{
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0)
        die("clock_gettime failed");
}
```

```
    return (double)ts.tv_sec * 1000.0 +
           (double)ts.tv_nsec / 1000000.0;
}

static void free_rgb(RGBImage *img)
{
    free(img->r);
    free(img->g);
    free(img->b);
    img->r = img->g = img->b = NULL;
}

static void free_gray(GrayImage *img)
{
    free(img->data);
    img->data = NULL;
}

static GrayImage alloc_gray(int width, int height)
{
    GrayImage img;
    img.width = width;
    img.height = height;
    img.data = xcalloc((size_t)width * (size_t)height, sizeof(double));
    return img;
}

/*
 * Read one whitespace-delimited PPM token while skipping comments.
 * This makes the P6 reader robust to comments anywhere in the header.
 */
static int read_token(FILE *f, char *buf, size_t buf_size)
{
    int c;
    size_t n = 0;

    do {
        c = fgetc(f);
        if (c == '#') {
            do {
                c = fgetc(f);
            } while (c != '\n' && c != EOF);
        }
    } while (c != EOF && (c == ' ' || c == '\t' ||
                          c == '\n' || c == '\r' || c == '\f'));

    if (c == EOF)
        return 0;

    do {
        if (n + 1 < buf_size)
            buf[n++] = (char)c;
        c = fgetc(f);
    } while (c != EOF && c != ' ' && c != '\t' &&
             c != '\n' && c != '\r' && c != '\f' && c != '#');

    buf[n] = '\0';

    if (c == '#') {
        do {
            c = fgetc(f);
        } while (c != '\n' && c != EOF);
    }

    return 1;
}

/* ----- PPM/PGM I/O ----- */

static RGBImage read_ppm(const char *path)
{
    FILE *f = fopen(path, "rb");
    if (!f) {
        fprintf(stderr, "Could not open input file: %s\n", path);
        exit(EXIT_FAILURE);
    }

    char token[64];

    if (!read_token(f, token, sizeof(token)) || strcmp(token, "P6") != 0)
        die("only binary PPM (P6) input is supported");

    if (!read_token(f, token, sizeof(token)))
        die("invalid PPM width");
    int width = atoi(token);

    if (!read_token(f, token, sizeof(token)))
        die("invalid PPM height");
    int height = atoi(token);

    if (!read_token(f, token, sizeof(token)))
        die("invalid PPM max value");
    int maxval = atoi(token);

    if (width <= 0 || height <= 0)
        die("invalid image dimensions");

    if (maxval != 255)
        die("only 8-bit PPM files (max value 255) are supported");

    size_t pixels = (size_t)width * (size_t)height;

    RGBImage img;
    img.width = width;
    img.height = height;
    img.r = xmalloc(pixels);
    img.g = xmalloc(pixels);
    img.b = xmalloc(pixels);

    unsigned char *buffer = xmalloc(3 * pixels);
}
```

```

size_t bytes_read = fread(buffer, 1, 3 * pixels, f);
fclose(f);

if (bytes_read != 3 * pixels) {
    free(buffer);
    free_rgb(&img);
    die("PPM pixel data is incomplete");
}

for (size_t i = 0; i < pixels; ++i) {
    img.r[i] = buffer[3 * i];
    img.g[i] = buffer[3 * i + 1];
    img.b[i] = buffer[3 * i + 2];
}

free(buffer);
return img;
}

static void write_pgm(const char *path, const GrayImage *img)
{
    FILE *f = fopen(path, "wb");
    if (!f) {
        fprintf(stderr, "Could not open output file: %s\n", path);
        exit(EXIT_FAILURE);
    }

    fprintf(f, "P5\n%d %d\n255\n", img->width, img->height);

    size_t pixels = (size_t)img->width * (size_t)img->height;
    unsigned char *buffer = xmalloc(pixels);

    for (size_t i = 0; i < pixels; ++i) {
        double value = img->data[i];

        if (value < 0.0) value = 0.0;
        if (value > 255.0) value = 255.0;

        buffer[i] = (unsigned char)lround(value);
    }

    if (fwrite(buffer, 1, pixels, f) != pixels) {
        free(buffer);
        fclose(f);
        die("Failed while writing PGM output");
    }

    free(buffer);
    fclose(f);
}

/* ----- Stage 1 ----- */

/*
 * RGB luminosity conversion.
 *
 * gray = 0.299 R + 0.587 G + 0.114 B
 */
static GrayImage rgb_to_grayscale(const RGBImage *img)
{
    GrayImage out = alloc_gray(img->width, img->height);

    size_t pixels = (size_t)img->width * (size_t)img->height;

    for (size_t i = 0; i < pixels; ++i) {
        out.data[i] =
            0.299 * (double)img->r[i] +
            0.587 * (double)img->g[i] +
            0.114 * (double)img->b[i];
    }

    return out;
}

/* ----- Convolution ----- */

/*
 * Generic naive convolution with zero padding.
 *
 * The reference paper describes its GPU convolution operations as
 * flipped convolutions. Therefore the kernel is accessed in reverse
 * order here as well.
 */
static GrayImage convolve(
    const GrayImage *img,
    const double *kernel,
    int kernel_size)
{
    int half = kernel_size / 2;

    GrayImage out = alloc_gray(img->width, img->height);

    for (int y = 0; y < img->height; ++y) {
        for (int x = 0; x < img->width; ++x) {

            double accumulator = 0.0;

            for (int ky = 0; ky < kernel_size; ++ky) {
                for (int kx = 0; kx < kernel_size; ++kx) {

                    int iy = y + ky - half;
                    int ix = x + kx - half;

                    double pixel = 0.0;

                    /* Zero-padding outside the image. */
                    if (iy >= 0 && iy < img->height &&
                        ix >= 0 && ix < img->width) {

                        pixel = img->data[
                            (size_t)iy * (size_t)img->width + (size_t)ix

```

```

];
    }

    int flipped_ky = kernel_size - 1 - ky;
    int flipped_kx = kernel_size - 1 - kx;

    accumulator +=
        pixel *
        kernel[flipped_ky * kernel_size + flipped_kx];
    }
}

out.data[
    (size_t)y * (size_t)img->width + (size_t)x
] = accumulator;
}
}

return out;
}

/* ----- Stage 2 ----- */

/*
 * Gaussian kernel shown in Equation 1 of the paper.
 *
 * The coefficients sum to 150, hence the normalization by 1/150.
 */
static const double GAUSSIAN_KERNEL[25] = {
    2, 4, 5, 4, 2,
    4, 9, 12, 9, 4,
    5, 12, 15, 12, 5,
    4, 9, 12, 9, 4,
    2, 4, 5, 4, 2
};

static GrayImage gaussian_blur(const GrayImage *img)
{
    double normalized[25];

    for (int i = 0; i < 25; ++i)
        normalized[i] = GAUSSIAN_KERNEL[i] / 150.0;

    return convolve(img, normalized, 5);
}

/* ----- Stage 3 ----- */

static const double SOBEL_X[9] = {
    1, 0, -1,
    2, 0, -2,
    1, 0, -1
};

static const double SOBEL_Y[9] = {
    1, 2, 1,
    0, 0, 0,
    -1, -2, -1
};

static void sobel(
    const GrayImage *img,
    GrayImage *magnitude,
    GrayImage *direction)
{
    GrayImage gx = convolve(img, SOBEL_X, 3);
    GrayImage gy = convolve(img, SOBEL_Y, 3);

    *magnitude = alloc_gray(img->width, img->height);
    *direction = alloc_gray(img->width, img->height);

    size_t pixels = (size_t)img->width * (size_t)img->height;

    for (size_t i = 0; i < pixels; ++i) {
        double x = gx.data[i];
        double y = gy.data[i];

        magnitude->data[i] = hypot(x, y);
        direction->data[i] = atan2(y, x);
    }

    free_gray(&gx);
    free_gray(&gy);
}

/* ----- Stage 4 ----- */

/*
 * Quantize the gradient direction into four orientations:
 *
 * 0 degrees
 * 45 degrees
 * 90 degrees
 * 135 degrees
 *
 * This is a standard Canny implementation choice. The paper describes
 * selecting the two neighboring pixels along the gradient direction,
 * but does not specify the complete numerical quantization rule.
 */
static GrayImage non_maximum_suppression(
    const GrayImage *magnitude,
    const GrayImage *direction)
{
    int w = magnitude->width;
    int h = magnitude->height;

    GrayImage out = alloc_gray(w, h);

    for (int y = 1; y < h - 1; ++y) {
        for (int x = 1; x < w - 1; ++x) {

```

```

int idx = y * w + x;

double angle =
    direction->data[idx] * 180.0 / M_PI;

if (angle < 0.0)
    angle += 180.0;

double current = magnitude->data[idx];
double neighbor1;
double neighbor2;

if ((angle >= 0.0 && angle < 22.5) ||
    (angle >= 157.5 && angle < 180.0)) {

    /* 0 degrees: left/right */
    neighbor1 = magnitude->data[idx - 1];
    neighbor2 = magnitude->data[idx + 1];

} else if (angle < 67.5) {

    /* 45 degrees: NE/SW */
    neighbor1 =
        magnitude->data[(y - 1) * w + (x + 1)];

    neighbor2 =
        magnitude->data[(y + 1) * w + (x - 1)];

} else if (angle < 112.5) {

    /* 90 degrees: up/down */
    neighbor1 =
        magnitude->data[(y - 1) * w + x];

    neighbor2 =
        magnitude->data[(y + 1) * w + x];

} else {

    /* 135 degrees: NW/SE */
    neighbor1 =
        magnitude->data[(y - 1) * w + (x - 1)];

    neighbor2 =
        magnitude->data[(y + 1) * w + (x + 1)];

}

if (current >= neighbor1 && current >= neighbor2)
    out.data[idx] = current;
else
    out.data[idx] = 0.0;

}

/*
 * The reference CUDA implementation uses a two-pixel border
 * exclusion in its suppression stage. We explicitly zero that
 * border in the sequential reference as well.
 */
for (int x = 0; x < w; ++x) {
    out.data[x] = 0.0;
    out.data[(h - 1) * w + x] = 0.0;

    if (h > 3) {
        out.data[w + x] = 0.0;
        out.data[(h - 2) * w + x] = 0.0;
    }

}

for (int y = 0; y < h; ++y) {
    out.data[y * w] = 0.0;
    out.data[y * w + (w - 1)] = 0.0;

    if (w > 3) {
        out.data[y * w + 1] = 0.0;
        out.data[y * w + (w - 2)] = 0.0;
    }

}

return out;
}

/* ----- Stage 5 ----- */

#define STRONG 255.0
#define WEAK 127.0

/*
 * Double threshold followed by 8-connected hysteresis.
 *
 * Hysteresis is implemented as a stack-based Flood Fill starting from
 * strong pixels. Weak pixels connected to a strong component become
 * strong; remaining weak pixels become non-edge.
 */
static GrayImage double_threshold_and_hysteresis(
    const GrayImage &suppressed,
    double low,
    double high)
{
    int w = suppressed->width;
    int h = suppressed->height;
    size_t pixels = (size_t)w * (size_t)h;

    GrayImage out = alloc_gray(w, h);

    /* Double threshold. */
    for (size_t i = 0; i < pixels; ++i) {
        double value = suppressed->data[i];

        if (value >= high)
            out.data[i] = STRONG;

```

```

        else if (value >= low)
            out.data[i] = WEAK;
        else
            out.data[i] = 0.0;
    }

    /*
     * Stack-based propagation from all strong pixels.
     * Each pixel is pushed at most once.
     */
    unsigned char *visited = xmalloc(pixels, sizeof(unsigned char));
    size_t *stack = xmalloc(pixels * sizeof(size_t));
    size_t stack_size = 0;

    for (size_t i = 0; i < pixels; ++i) {
        if (out.data[i] == STRONG) {
            visited[i] = 1;
            stack[stack_size++] = i;
        }
    }

    while (stack_size > 0) {
        size_t idx = stack[--stack_size];

        int y = (int)(idx / (size_t)w);
        int x = (int)(idx % (size_t)w);

        for (int dy = -1; dy <= 1; ++dy) {
            for (int dx = -1; dx <= 1; ++dx) {

                if (dx == 0 && dy == 0)
                    continue;

                int ny = y + dy;
                int nx = x + dx;

                if (ny < 0 || ny >= h || nx < 0 || nx >= w)
                    continue;

                size_t nidx =
                    (size_t)ny * (size_t)w + (size_t)nx;

                if (!visited[nidx] && out.data[nidx] == WEAK) {
                    out.data[nidx] = STRONG;
                    visited[nidx] = 1;
                    stack[stack_size++] = nidx;
                }
            }
        }
    }

    /* Any weak pixel not connected to a strong pixel is discarded. */
    for (size_t i = 0; i < pixels; ++i) {
        if (out.data[i] == WEAK)
            out.data[i] = 0.0;
    }

    free(visited);
    free(stack);

    return out;
}

/* ----- Main pipeline ----- */

int main(int argc, char **argv)
{
    if (argc < 3 || argc > 5) {
        fprintf(stderr,
            "Usage: %s input.ppm output.pgm [low] [high]\n",
            argv[0]);
        return EXIT_FAILURE;
    }

    const char *input_path = argv[1];
    const char *output_path = argv[2];

    double low = (argc >= 4) ? atof(argv[3]) : 50.0;
    double high = (argc >= 5) ? atof(argv[4]) : 100.0;

    if (low < 0.0 || high < 0.0 || low > high)
        die("thresholds must satisfy 0 <= low <= high");

    /*
     * I/O is deliberately outside the algorithm timing.
     * This makes the reported time represent the Canny computation,
     * rather than file loading/writing.
     */
    RGBImage rgb = read_ppm(input_path);

    double t0, t1;
    double t_gray, t_blur, t_sobel;
    double t_nms, t_threshold;

    /* Stage 1 */
    t0 = now_ms();
    GrayImage gray = rgb_to_grayscale(&rgb);
    t1 = now_ms();
    t_gray = t1 - t0;

    /* Stage 2 */
    t0 = now_ms();
    GrayImage blurred = gaussian_blur(&gray);
    t1 = now_ms();
    t_blur = t1 - t0;

    /* Stage 3 */
    t0 = now_ms();
    GrayImage magnitude;
    GrayImage direction;
    sobel(&blurred, &magnitude, &direction);

```

```

t1 = now_ms();
t_sobel = t1 - t0;

/* Stage 4 */
t0 = now_ms();
GrayImage suppressed =
    non_maximum_suppression(&magnitude, &direction);
t1 = now_ms();
t_nms = t1 - t0;

/* Stage 5 */
t0 = now_ms();
GrayImage edges =
    double_threshold_and_hysteresis(&suppressed, low, high);
t1 = now_ms();
t_threshold = t1 - t0;

double total =
    t_gray + t_blur + t_sobel + t_nms + t_threshold;

write_pgm(output_path, &edges);

printf("Resolution: %dx%d\n", rgb.width, rgb.height);
printf("Thresholds: low=%.2f, high=%.2f\n", low, high);
printf("\n");
printf("%-28s %12s\n", "Stage", "Time (ms)");
printf("-----\n");
printf("%-28s %12.3f\n", "Grayscale", t_gray);
printf("%-28s %12.3f\n", "Gaussian Blur", t_blur);
printf("%-28s %12.3f\n", "Sobel Filter", t_sobel);
printf("%-28s %12.3f\n", "NMS", t_nms);
printf("%-28s %12.3f\n", "Threshold + Hysteresis", t_threshold);
printf("-----\n");
printf("%-28s %12.3f\n", "Total", total);

free_rgb(&rgb);
free_gray(&gray);
free_gray(&blurred);
free_gray(&magnitude);
free_gray(&direction);
free_gray(&suppressed);
free_gray(&edges);

return EXIT_SUCCESS;
}

```

REFERENCES

- [1] M. Horvath Jr., M. Bowers, and S. Alawneh, "Canny Edge Detection on GPU using CUDA," in *2023 IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC)*, 2023, pp. 419–425, doi: 10.1109/CCWC57344.2023.10099273.
- [2] T. L. Ben Cheikh, G. Beltrame, G. Nicolescu, F. Cheriet, and S. Tahar, "Parallelization strategies of the canny edge detector for multi-core CPUs and many-core GPUs," in *2012 10th IEEE International NEWCAS Conference*, Montreal, QC, Canada, 2012, pp. 49–52.
- [3] S. Rahamneh and L. Sawalha, "An OpenCL-Based Acceleration for Canny Algorithm Using a Heterogeneous CPU-FPGA Platform," in *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2019, p. 322.
- [4] Y. Luo and R. Duraiswami, "Canny edge detection on NVIDIA CUDA," in *2008 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, 2008, pp. 1–8.